

Unit III: Functions and Strings

Section 1: Functions in Python

Functions are **blocks of code that perform a specific task**. Using functions makes code **organized, reusable, and easier to debug**.

1.1 Need for Functions

- Reduce **code repetition** → write once, use multiple times.
- Break complex programs into **smaller manageable parts**.
- Make **programs easier to read and maintain**.

Example: Instead of writing repeated code to calculate the square of a number:

```
# Without function
print(5**2)
print(10**2)
print(3**2)
```

With function:

```
def square(x):
    return x ** 2

print(square(5))
print(square(10))
print(square(3))
```

1.2 Function Definition and Call

- **Definition:** Create a function using `def` keyword.
- **Call:** Execute the function by its name.

Syntax:

```
def function_name(parameters):
    """Documentation string (optional)"""
    # statements
    return value # optional
```

Example:

```
def greet(name):  
    """Function to greet a person"""  
    print("Hello", name)  
  
greet("Alice") # Output: Hello Alice
```

1.3 Variable Scope and Lifetime

- **Scope:** Region of the program where a variable is accessible.
 - **Local variables:** Defined inside function → accessible only inside function.
 - **Global variables:** Defined outside all functions → accessible anywhere in the program.
- **Lifetime:** Duration a variable exists in memory. Local variables exist only during function execution.

Example:

```
x = 10 # Global variable  
  
def test():  
    y = 5 # Local variable  
    print(x, y)  
  
test() # Output: 10 5  
print(y) # Error: y is not defined
```

1.4 Return Statement

- Used to **send a value back** from the function.

Example:

```
def add(a, b):  
    return a + b  
  
result = add(5, 10)  
print(result) # Output: 15
```

1.5 Lambda or Anonymous Functions

- Small one-line functions without a name.
- **Syntax:** lambda arguments: expression

Example:

```
square = lambda x: x ** 2
```

```
print(square(5)) # Output: 25
```

```
add = lambda a, b: a + b  
print(add(3, 4)) # Output: 7
```

1.6 Documentation String (Docstring)

- Used to **describe what the function does**.
- Accessed with `.__doc__`

Example:

```
def greet(name):  
    """This function greets the person with their name"""  
    print("Hello", name)
```

```
print(greet.__doc__) # Output: This function greets the person with their  
name
```

1.7 Good Programming Practices

- Use **meaningful names** for functions and variables.
 - Keep functions **short and focused on one task**.
 - Include **docstrings** for clarity.
 - Avoid **deeply nested loops or conditions** inside functions.
-

1.8 Modules and Packages

- **Module:** A file containing Python definitions (functions, variables, classes).
 - Example: `math.py` with math functions.
- **Package:** A **collection of modules** organized in folders.
 - Example: `numpy` package contains modules like `numpy.linalg`, `numpy.fft`.
- **Importing modules:**

```
import math  
print(math.sqrt(16)) # Output: 4.0
```

- **Standard library modules:** Built-in Python modules like `math`, `random`, `os`, `datetime`.
-

Section 2: Strings in Python

Strings are **sequences of characters** enclosed in single ' or double " quotes.

2.1 String Operations

- **Concatenation:** Join strings using +

```
a = "Hello"
b = "World"
print(a + " " + b)  # Output: Hello World
```

- **Appending:** Add strings at the end using +=

```
a = "Hello"
a += " World"
print(a)  # Output: Hello World
```

- **Multiplication:** Repeat strings using *

```
print("Hi! " * 3)  # Output: Hi! Hi! Hi!
```

- **Slicing:** Access part of string using indices [start:end:step]

```
s = "Python"
print(s[0:4])  # Output: Pyth
print(s[::2])  # Output: Pto
```

2.2 String Immutability

- Strings **cannot be changed** once created. Any operation creates a **new string**.

```
s = "Hello"
s[0] = "h"  # Error
s = "h" + s[1:]  # Correct way to modify
print(s)  # Output: hello
```

2.3 String Formatting

- Use % operator or format() method to insert variables into strings.

```
name = "Alice"
age = 20
print("My name is %s and I am %d years old" % (name, age))
# Output: My name is Alice and I am 20 years old

print("My name is {} and I am {} years old".format(name, age))
```

```
# Output: My name is Alice and I am 20 years old
```

2.4 Built-in String Methods and Functions

Method	Description	Example
<code>upper()</code>	Converts to uppercase	<code>"hello".upper()</code> → <code>"HELLO"</code>
<code>lower()</code>	Converts to lowercase	<code>"HELLO".lower()</code> → <code>"hello"</code>
<code>strip()</code>	Removes spaces	<code>" hi ".strip()</code> → <code>"hi"</code>
<code>replace(old, new)</code>	Replace substring	<code>"hi hi".replace("hi", "bye")</code> → <code>"bye bye"</code>
<code>split(separator)</code>	Split string into list	<code>"a,b,c".split(",")</code> → <code>['a', 'b', 'c']</code>
<code>join(list)</code>	Join list into string	<code>" ".join(['a', 'b'])</code> → <code>"a b"</code>
<code>len()</code>	Length of string	<code>len("Python")</code> → <code>6</code>

2.5 `ord()` and `chr()` Functions

- `ord(char)` → Returns ASCII code of character
- `chr(code)` → Returns character of ASCII code

```
print(ord('A')) # Output: 65
print(chr(65))  # Output: 'A'
```

2.6 `in` / `not in` Operators

- Check if substring exists in a string.

```
s = "Python"
print("Py" in s)      # True
print("Java" not in s) # True
```

2.7 Comparing Strings

- Strings can be compared using `==`, `!=`, `<`, `>` based on **lexicographic order** (dictionary order).

```
print("apple" < "banana") # True
print("abc" == "ABC")     # False
```

2.8 Iterating Strings

- Strings can be **looped through character by character**.

```
s = "Python"
for ch in s:
    print(ch)
```

2.9 String Module

- Python has a built-in **string module** for common string constants and utility functions.

```
import string
print(string.ascii_letters) #
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
print(string.digits)       # 0123456789
```

Key Takeaways for Students

- Functions **make code reusable and organized**.
- Lambda functions are **short anonymous functions**.
- Modules and packages allow **code modularity and reuse**.
- Strings are **immutable sequences** with many useful methods for manipulation.
- Proper string operations, formatting, and iteration are **fundamental skills in Python**.